Security Auditing for Python Cryptographic Code

Rounds 1–6 Training, Data Governance, and Evaluation Report

AI-Assisted Cryptographic Code Security Auditor

2026-10-05

Abstract

This report documents a research prototype for auditing cryptographic misuse in Python. The system uses 13 in-house Semgrep CRYPTO rules as a static candidate generator and a QLoRA fine-tuned Qwen3-4B as the semantic discriminator, covering two tasks: **detect** (decide at function level whether the code is vulnerable, and emit a CWE and severity) and **triage** (given code plus a static finding, emit Confirm/Reject, an explanation, and an optional patch). Rounds 1–4 established the two-task baseline, exposed catastrophic forgetting from incremental continued training, recovered through full fresh-base retraining, and tested synthetic negative-sample scaling. Round 5 produced a dataset QA failure and a clear model regression. Round 6 folded real-data strengthening, rule fixtures, strong-model pre-review, and browser-based human review into a single full retrain. The shipped R6 artifacts contain 4,122 train/validation/test records, of which 401 are new and 79 are real (30 unique verified cryptographic positives); the frozen holdout is 19 detect and 36 triage records (only 4 Confirm). R6 recovers from the R5 detect collapse, reaching F1 0.588 on the frozen holdout and 0.671 on the design test split, but it fails the precision, FPR, and 18-probe accuracy gates. This report therefore defines R6 as a **reproducible research baseline**, not a production-grade auditor. It deliberately preserves the Round 6 candidate harvesting, cost estimation, K3 binary review, browser human review, provenance tagging, dataset freezing, and failure analysis for later review and for R7 design.

Contents

1 Project Scope and Purpose **3**

1.1 One-line summary . . . 3

1.2 The two tasks . . . 3

1.3 Scope and non-goals . . . 3

2 Background: Cryptography, CWE, and Code Auditing **4**

2.1 Common cryptographic building blocks . . . 4

2.2 CWE taxonomy and the 13 CRYPTO rules . . . 4

2.3 SAST, Semgrep, AST, and false positives . . . 5

2.4 How detect and triage differ . . . 6

2.5 LLM fine-tuning basics . . . 6

2.6 Evaluation metrics . . . 6

3 System Architecture, Data Sources, and Provenance Tagging **7**

3.1 Audit pipeline . . . 7

3.2 Data layers . . . 7
3.3 R6 final data snapshot . . . 7
3.4 Data-composition risk: class balance is not effective signal . . . 7
3.5 Leakage audit . . . 8

4 Rounds 1–5: How the Problems Emerged 9
4.1 Round 1: two-task baseline . . . 9
4.2 Round 2: triage incremental training and catastrophic forgetting . . . 9
4.3 Round 3: fresh-base full merge retrain . . . 10
4.4 Round 4: negative-sample scaling and real-project validation . . . 10
4.5 Round 5: dataset QA failure and model regression . . . 11

5 Round 6: Data Optimisation, Strong-Model Review, and Human Review 12
5.1 R6 repair goals . . . 12
5.2 The 322 new curated/rule-fixture records . . . 12
5.3 Secure-library hedging and the reference layer . . . 13
5.4 The real candidate pool: from advisory to function-level code pair . . . 13
5.5 Strong-model pre-review: budget, model choice, and strict binary output . . . 13
5.6 The browser-based human review platform . . . 14
5.7 Pre-training freeze and reproducibility constraints . . . 15

6 R6 Training and Reproducibility Information 15
6.1 Model and platform . . . 15
6.2 Hyperparameters and configuration differences . . . 16

7 R6 Evaluation Results 16
7.1 Evaluation suites . . . 16
7.2 Frozen holdout . . . 16
7.3 The 18 real triage probes . . . 17
7.4 R6 design test split . . . 17
7.5 13-rule capability matrix . . . 17
7.6 Secure-library anti-false-positive . . . 17
7.7 Acceptance gates . . . 18

8 R6 Versus R5: What Recovered, What Did Not 18
8.1 Recovery . . . 18
8.2 Remaining problems . . . 18

9 Conclusion, Limitations, and the R7 Roadmap 18
9.1 Conclusion . . . 18
9.2 Main limitations . . . 19

9.3 R7 roadmap 19

10 Review Q&A and Interview Points **20**

10.1 Frequently asked questions 20

10.2 Things to watch during the experiments 20

A Reproduction Commands and Key Files **21**

A.1 Build and freeze 21

A.2 Evaluation 21

A.3 Key artifacts 21

B Glossary **22**

1 Project Scope and Purpose

1.1 One-line summary

This project studies a composite code-audit prototype: static analysis finds suspicious cryptographic API usages with high recall; a small domain model decides whether a finding is real, classifies the CWE, explains the reason, and proposes a fix direction; uncertain records go to a strong model or to browser-based human review. The system ultimately emits structured JSON and a Markdown audit report.

The long-term product hypothesis is a "security-audit copilot": it does not replace a security engineer, rather it reduces manual triage time and keeps the audit rationale, data provenance, and model verdicts as traceable evidence.

1.2 The two tasks

Table 1: The system's two audit tasks

Task	Input	Output
detect	one Python function or code snippet	`vulnerable`, `cwe`, `severity`, `confidence`, `explanation`
triage	code + a static-analysis finding	`cwe`, `severity`, `verdict=Confirm|Reject`, `explanation`, optional `patch`

1.3 Scope and non-goals

The current version covers only the defined cryptographic-misuse patterns: weak hashing, broken algorithms, AES-ECB, predictable IV/nonce, hard-coded keys, weak randomness, weak key length, insecure TLS, plaintext passwords, weak KDF parameters, and non-constant-time comparison. It does not claim to discover unknown classes of cryptographic vulnerabilities, and it does not generate attacks or exploit code.

This report explicitly does not claim any of the following:

- that the model reaches production-grade precision, recall, or CWE routing;

- that strong-model K3 review was rewritten into human ground truth;
- that the merged training artifacts are leak-free;
- that R6's test-split boundary values represent true cryptographic generalisation;
- that "training longer" is a solution to the current bottleneck.

2 Background: Cryptography, CWE, and Code Auditing

2.1 Common cryptographic building blocks

Auditing cryptographic code requires separating "the algorithm itself" from "how the algorithm is used in the business logic". A weak primitive is not automatically a vulnerability: MD5 for password storage is usually a defect, whereas MD5 for a cache key or change detection usually is not. This context boundary is exactly what the model finds hardest to learn.

Table 2: Cryptographic building blocks and their audit concerns

Building block	Role	Common audit risks
Symmetric encryption	Encrypt/decrypt with one key, e.g. AES	ECB mode, fixed IV, nonce reuse, weak key length
AEAD	Confidentiality plus integrity, e.g. AES-GCM, ChaCha20-Poly1305	Nonce reuse breaks authentication; wrong key origin and nonce lifetime
Asymmetric encryption and signatures	Public-key encryption, private-key signing, e.g. RSA, ECDSA, Ed25519	Small RSA modulus, wrong padding, weak key or randomness source
Hashing	Map arbitrary input to a fixed-length digest, e.g. SHA-256	MD5/SHA-1 for security-sensitive data; mistaking a plain digest for a password hash
MAC/HMAC	Keyed message integrity and authenticity	Non-constant-time comparison, key reuse, algorithm downgrade
KDF and password hashing	Derive keys or resist offline cracking, e.g. PBKDF2, scrypt, Argon2	Too few iterations, missing salt, attacker-controlled parameters
Randomness	Generate keys, tokens, salts, IVs, nonces	Using `random` (Mersenne Twister) instead of `secrets` or `os.urandom`
TLS and certificates	Verify server identity and establish an encrypted channel	`verify=False`, `CERT_NONE`, hostname check disabled, trust root overridden

IV (initialization vector) and nonce (number used once) are prerequisites for the security of many modes. CBC-like modes require an unpredictable IV; streaming AEAD such as GCM requires that a nonce never repeat under the same key. Merely seeing "the function uses AES" is not enough to judge safety; the mode, IV/nonce, key origin, and intended use must all be checked.

Password hashing must not be confused with general-purpose hashing. A bare `sha256(password)` is fast, which is fine for file integrity but unsuitable for resisting offline cracking. A password system should use a salted, cost-tunable KDF or password hash; modern choices include Argon2id, scrypt, bcrypt, or a standard-conforming PBKDF2 configuration. The audit must also judge whether the iteration count is too low, whether the salt is random and unique, and whether the parameters can be attacker-controlled.

2.2 CWE taxonomy and the 13 CRYPTO rules

The project binds static rules to CWE targets. A CWE is a defect class; a rule is an engineering-detectable pattern; the two are not identical. One rule can cover several CWEs and one CWE

may need several rules.

Table 3: The 13 CRYPTO rule families and their core CWEs

Rule	Problem	Target CWE
CRYPTO-001	Weak digest (MD5 etc.) in a security-sensitive context	CWE-327 / CWE-916
CRYPTO-002	Weak digest (SHA-1 etc.) in a security-sensitive context	CWE-327 / CWE-916
CRYPTO-003	DES or 3DES	CWE-327
CRYPTO-004	RC4	CWE-327
CRYPTO-005	AES-ECB	CWE-327
CRYPTO-006	Predictable or reused IV/nonce	CWE-329
CRYPTO-007	Hard-coded key or password	CWE-321
CRYPTO-008	Weak pseudo-random number generator	CWE-338
CRYPTO-009	Weak key length or weak curve	CWE-326
CRYPTO-010	Insecure TLS or certificate validation	CWE-295
CRYPTO-011	Plaintext password storage or logging	CWE-256
CRYPTO-012	Weak KDF parameters	CWE-916
CRYPTO-013	Non-constant-time comparison	CWE-208

The report focuses on nine target CWEs: CWE-321 (hard-coded key), CWE-326 (inadequate encryption strength), CWE-327 (broken or risky algorithm), CWE-329 (IV/nonce not randomized), CWE-338 (weak PRNG), CWE-916 (insufficient password-hash computational effort), CWE-208 (observable timing discrepancy), CWE-295 (improper certificate validation), and CWE-256 (plaintext storage of a password). The R6 candidate pool also surfaces neighbouring classes such as CWE-798 (hard-coded credentials), CWE-330 (insufficient randomness), and CWE-328 (reversible one-way hash), but neighbouring classes must not enter the target training set without a rule mapping.

2.3 SAST, Semgrep, AST, and false positives

SAST (static application security testing) analyses source code without running the program. Semgrep produces findings through pattern and lightweight semantic matching; the Python AST extracts functions, compares structure, locates calls, and detects near-duplicates.

The strength of static analysis is speed, interpretability, and precise location; its weakness is that a pattern hit is not the same as a real defect. For example:

- a crypto library implements MD5 or DES for legacy-protocol or historical-format compatibility, which is a "valid false positive";
- test fixtures, cache keys, ETags, consistency checks, sharding, and jitter use weak primitives usually without a security objective;
- a function does call AES but uses a fixed IV, ECB, or a predictable nonce, which is still a real problem.

The system therefore adopts "static recall plus LLM semantic routing": the static rules act as a candidate generator, and the LLM judges purpose and security boundary. The LLM can reduce false positives, but it can also hallucinate cache, test, or ETag context that is absent from the code, so a structured schema, an evaluation set, and provenance auditing are mandatory.

2.4 How detect and triage differ

detect is an open judgement: the model sees only the code, decides whether it is vulnerable, and gives a CWE. triage is a constrained judgement: the model sees both the code and one specific finding, and decides Confirm or Reject. Their supervision signals differ:

- detect needs safe code, real vulnerabilities, and both rule-hit and rule-miss negatives;
- triage needs Reject cases (rule hit but no security purpose) and Confirm cases (rule hit with a real security-sensitive purpose);
- the detect training distribution controls whether the model "over-reports everything";
- the triage training distribution controls whether the model over-generalises weak-primitive usages into Reject.

2.5 LLM fine-tuning basics

This project uses supervised fine-tuning (SFT) and QLoRA. SFT treats the input prompt and the expected JSON output as a training pair; LoRA freezes the base weights and trains only low-rank adapter matrices; QLoRA further applies 4-bit quantisation to reduce memory. The core hyperparameters are:

Table 4: Fine-tuning terminology

Term	Meaning and effect
rank (r)	Rank of the LoRA low-rank matrices; larger means more capacity and more risk of overfitting or disturbing old abilities
alpha	LoRA scaling coefficient; usually proportional to rank; R1–R6 mainly use rank=16, alpha=32
dropout	Random-dropout rate of the adapter layers; used to suppress overfitting
learning rate	Update step size; too large destroys base abilities, too small learns no new boundary
epochs	Number of passes over the data; more is not better and can reinforce templates and noise
max length	Truncation length; over-long samples are truncated, losing code context
fresh-base retrain	Retrain from the original base with the full merged dataset; the most stable paradigm in this project
incremental update	Continue training the existing model on new samples only; R2 and R4 both triggered catastrophic forgetting
catastrophic forgetting	After learning a new task, performance on old tasks degrades significantly

Catastrophic forgetting recurred throughout the project. R2 fed only 54 triage increments, dropped probe FPR to 0, but pushed detect CWE accuracy from 0.796 down to 0.433 and domain Confirm recall from 0.357 down to 0. R4 micro-continued training was worse: detect recall fell from 0.990 to 0.584 and domain Confirm recall from 0.929 to 0.071. The engineering constraint that follows is: **when tasks or distributions change noticeably, do a fresh-base full merge retrain; incremental continued training is only for small repairs on the same distribution.**

2.6 Evaluation metrics

Let TP, FP, FN, and TN denote true positive, false positive, false negative, and true negative. The main detect metrics are:

$$\text{precision} = \frac{TP}{TP + FP}, \qquad \text{recall} = \frac{TP}{TP + FN}, \tag{1}$$

$$F1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}, \qquad \text{accuracy} = \frac{TP + TN}{TP + FP + FN + TN}. \tag{2}$$

CWE accuracy is computed only over ground-truth positives and asks whether the CWE is exactly correct once the code has been judged vulnerable. triage Confirm recall is the fraction of true Confirm cases retained; FPR (false positive rate) is the fraction of true Reject cases that are wrongly Confirmed:

$$\text{FPR} = \frac{FP}{FP + TN}. \tag{3}$$

Small positive counts make recall extremely unstable. The frozen triage set has only 4 Confirm cases, so a single prediction flip moves recall by 25 points; this report therefore treats frozen Confirm recall as a count, not an acceptance gate. Likewise, 30 real cryptographic positives is below the project's pre-set 50–100 threshold, so aggregate scores cannot be used to claim real-world cryptographic generalisation.

3 System Architecture, Data Sources, and Provenance Tagging

3.1 Audit pipeline

Python source → 13 Semgrep CRYPTO rules → candidate finding → Qwen3-4B fine-tuned model (detect / triage) → structured JSON → Markdown audit report

The main modules are `rules/`, `analyzer/`, `agent/`, `model/`, `evaluation/`, `scripts/`, and `reviewer/`. Training and inference are governed by the same prompt templates, so that the ChatML training data and the deployment input distribution stay consistent.

3.2 Data layers

3.3 R6 final data snapshot

R6 merges real records directly into the sixth round rather than creating a separate Round 7. The final uploaded set is:

R6 adds 401 records relative to the R5 base:

Of the 79 real records, 60 come from K3 model review and 19 from browser human review. Model-reviewed records keep `review_mode=model` and `model=kimi-k3`; human records keep `review_mode=manual`. Even though the project permits K3 labels as final labels, the report must not describe them as human review.

The CWE distribution of real detect positives is: CWE-295 (12), CWE-208 (9), CWE-327 (3), CWE-338 (2), CWE-321 (2), CWE-326 (1), CWE-256 (1). The real triage export is 30 Confirm / 19 Reject, with the signal concentrated in CRYPTO-010 (12 Confirm) and CRYPTO-013 (9 Confirm).

3.4 Data-composition risk: class balance is not effective signal

R6 detect has 2,037 positives and 1,596 negatives; the surface ratio looks reasonable, but only 434 positives belong to target crypto CWEs and the other 1,603 are general Web/application/memory

Table 5: Semantic layers of the training data

Layer	Typical source	Use and limitation
Public vulnerability data	PyCode-Vul	Provides general Python detect signal; most entries are Web/application bugs, not crypto-specific truth
Domain rule fixtures	Semgrep, Bandit test cases	Provide rule hits, safe alternative forms, Reject/negatives; `verified=false`, not real vulnerability samples
Manually/model-constructed samples	curated, combination templates	Fill the 13 rule families and class ratios; may be templated, so overfitting must be prevented
Real advisory records	GHSA, OSV/PyPI, PyPA, fix commit	Vulnerability and fix function pairs with a provenance chain; currently the most valuable real crypto signal
Secure reference	Fixed-commit secure-library code	Used for fix suggestions and safe-code comparison; does not enter training unless it passes a review gate

Table 6: R6 final data split

Split	detect	triage	Total
train	2732	383	3115
val	299	48	347
test	602	58	660
Total	**3633**	**489**	**4122**

vulnerabilities. In other words, most positive supervision is spent on non-crypto defects. The per-CWE sample counts are:

Per-class recall on rare CWEs will have very high variance. If training samples randomly over the total, rare classes and real advisory positives are diluted further; future work needs a train-only crypto-focus sampler with minimum quotas by CWE and rule.

3.5 Leakage audit

The goal of the frozen evaluation set is to remove every sample whose whitespace-normalised code and AST are identical to something in the final R5/R6 train or val set. The frozen R6 evaluation set is:

- detect: 19 records (7 positive / 12 negative), 0 removed;
- triage: 36 records (4 Confirm / 32 Reject), with 6 records removed as duplicates of `round4-real-passlib`.

However, a cross-split audit of the merged artifacts still finds residual duplicates inherited from R4/R5:

It is therefore only defensible to claim that "the frozen evaluation set is not exposed to the optimiser-visible set", not that the entire merged training set is leak-free. Future additions must keep splitting by repository and commit, not randomly by row.

Table 7: R6 added-record provenance

Source layer	Count	verified and use
curated, rule fixtures, and supplements	322	All `verified=false`; used for rule coverage, negatives/Reject, and tool regression
Real advisory detect records	30	All vulnerability positives; 23 by K3, 7 by human
Real advisory triage records	49	30 Confirm / 19 Reject; 37 by K3, 12 by human
Total	**401**	79 real records; unique real crypto positives still only 30

Table 8: Target crypto CWE counts among detect positives

CWE	Positives	CWE	Positives
CWE-327	275	CWE-916	11
CWE-326	63	CWE-256	10
CWE-295	33	CWE-321	5
CWE-208	18	CWE-329	5
CWE-338	14		

4 Rounds 1–5: How the Problems Emerged

4.1 Round 1: two-task baseline

Goal: first domain adaptation on public Python vulnerability data plus domain crypto data, running the full detect/triage chain.

Data: detect 2,406 + triage 132 = 2,538 records.

Key configuration: LoRA rank=8, learning rate 1e-4.

Results:

- detect CWE accuracy 0.796, F1 0.656;
- domain Confirm recall 0.357;
- 18 triage probes: Confirm recall 0.917, FPR 0.667, accuracy 0.722.

Exposed problem: the model was usable on general detect, but triage Confirmed many non-security uses of weak primitives. Typical false positives included MD5 in cache keys, SHA-1 in change detection, `random` in backoff jitter, test fixtures, and legacy-compatible implementations. FPR 0.667 shows that "rule hit" was wrongly treated as "real security defect".

4.2 Round 2: triage incremental training and catastrophic forgetting

Goal: add triage increments only, to push down FPR.

Data: 54 records (28 Confirm + 26 Reject), triage only.

Key configuration: batch=8, learning rate 1e-4, 3 epochs, rank=16 / alpha=32 / dropout=0.05, max length=1024.

Results:

- probe FPR fell from 0.667 to **0.000** and accuracy rose from 0.722 to 0.944;

Table 9: Residual near-duplicates in the merged training artifacts

Pair	Exact duplicate rows	Identical-AST rows
train / val	2	21
train / test	2	17
val / test	2	2

- detect CWE accuracy fell from 0.796 to **0.433**, with 343/569 samples shifting CWE prediction;
- domain Confirm recall fell from 0.357 to **0.000**, with the model judging almost everything Reject.

Lesson: feeding only new-task increments destroys the old-task representation. A short-term improvement in a target metric is not an overall capability gain. Incremental continued training was from then on listed as a high-risk route.

4.3 Round 3: fresh-base full merge retrain

Goal: abandon R2 continued training, retrain from the base on the full merged dataset, and restore both detect and triage.

Data: 2,595 = R1's 2,538 + R2's 54 + 3 signature/authentication repairs.

Key configuration: batch=16, learning rate 1e-4, 3 epochs, rank=16 / alpha=32 / dropout=0.005, cosine, max length=2048.

Key decision: append `TRIAGE_PURPOSE_GUIDE` to the triage prompt, requiring the model to judge explicitly whether the flagged call is for a security or a non-security purpose. Without the guide the same model had FPR 0.500; with the guide FPR was 0.000 while Confirm recall stayed 1.000.

Results:

- detect recall 0.990, precision 0.537, F1 0.697, CWE accuracy 0.921;
- domain Confirm recall 0.929, FPR 0.067, accuracy 0.931;
- 18 probes: accuracy 18/18.

Leftover problem: detect still reported almost everything, 562/569 judged vulnerable, with `tp/fp/fn/tn = 302/260/3/4`. Precision 0.537 is nearly the positive prior 0.536, showing that the binary vulnerable decision lacks discriminative power. The root cause is that detect negatives were almost all non-crypto Django code, so the model had never seen "secure cryptographic code".

4.4 Round 4: negative-sample scaling and real-project validation

Goal: add secure cryptographic negatives, reduce detect over-reporting, and hold out real crypto libraries for validation.

Negative-sample design:

- Class A: modern secure patterns such as scrypt, Argon2, bcrypt, PBKDF2, AES-GCM with random nonce, ChaCha20-Poly1305, SHA-3, HKDF, ECDH, `secrets/os.urandom`, RSA-OAEP, Ed25519;

- Class B: weak primitives used for non-security purposes such as cache keys, deduplication, sharding, change detection, backoff jitter, and test fixtures;
- strong-B: rules do fire, but the model must judge non-vulnerable from the purpose;
- weak-B: rules do not fire; used as ordinary safe negatives.

Two routes:

Table 10: R4 incremental continued training vs full retrain

	Micro continued training	**Full fresh-base**
Start point	R3 model	original base
Data	77 negatives	3,315 (detect 3126 + triage 189)
Negatives	42 A + 35 B	added 150 positives + 493 negatives, final 1563:1563
lr / epochs	5e-5 / 2	1e-4 / 3

Results:

- full 50:50: detect recall 0.984, precision 0.536, F1 0.694, CWE accuracy 0.897; fp=260, tn=4 basically unchanged from R3;
- micro continued training: detect recall 0.990 → 0.584, domain Confirm recall 0.929 → 0.071, catastrophic forgetting again;
- 42 real-project validation: detect judged only 4/7 safe functions correctly; domain triage FPR was 0.429 (micro) / 0.571 (full); all 11 passlib legacy-hash cases were wrongly judged Confirm.

Lesson: the number of synthetic negatives did not teach the model the security boundary of real libraries. Templated negatives are easily memorised and cannot replace real library code and protocol-compatibility scenarios.

4.5 Round 5: dataset QA failure and model regression

R5 was meant to keep repairing the data distribution and triage ability, but the round produced a clear regression. The submitted R5 run on the detect slice produced `tp/fp/fn/tn = 0/2/3/8`, F1 0.000, accuracy 0.615; triage n=41, 3 Confirm / 38 Reject, FPR 0.368. The R5 dataset QA report is FAIL and includes:

- `backtest_pos_missing=22`: samples that should have a positive back-test signal are missing;
- `backtest_bneg_no_fire=25`: strong-B negatives did not fire the expected rule;
- the same configuration re-run produced detect `0/1/3/9` and triage FPR 0.289, showing run-to-run variance in the point estimate.

Lesson: the data-quality gate must precede training. R5's regression was part data problem and part run variance; a single metric must not be treated as stable capability, and bad data must not enter the next round unfixed.

5 Round 6: Data Optimisation, Strong-Model Review, and Human Review

5.1 R6 repair goals

R6 was designed against the four R5 problems:

Table 11: R5 problems and R6 countermeasures

R5 problem	R6 countermeasure	Result
Triage Confirm recall decline	Add Confirm samples, keep binary review	R6 adds triage Confirm 113 / Reject 72
Domain FPR rebound	Scale Reject and hedge with secure libraries	Frozen FPR 0.125, still above the 0.06 target
Triage class imbalance	Add samples by rule family and type	All 13 rule families covered
Insufficient detect rule coverage	Add rule fixtures and real advisory records	13/13 rule families covered; 30 real positives still low

5.2 The 322 new curated/rule-fixture records

The 322 baseline additions come from 8 fixed source files:

Table 12: Sources of the 322 R6 baseline additions

Source file	Count
`new_detect.jsonl`	33
`new_triage.jsonl`	21
`supplement_detect.jsonl`	28
`supplement_triage.jsonl`	5
`supplement_v2_detect.jsonl`	21
`supplement_v2_triage.jsonl`	6
`rule_fixture_detect.jsonl`	104
`rule_fixture_triage.jsonl`	104
Total	**322**

Detect is 216 records (130 positive / 86 negative) and triage is 185 records (113 Confirm / 72 Reject) once real records are included; within the 322 baseline, detect is 186 (100 pos / 86 neg) and triage is 136 (83 Confirm / 53 Reject).

The rule fixtures mainly come from:

- Semgrep official rule tests, using `\# ruleid:` / `\# ok:` markers; licensed under the Semgrep Rules License v1.0;
- Bandit example programs, licensed under Apache-2.0;
- only rules semantically consistent with CRYPTO-001 through CRYPTO-013 are mapped; broader patterns such as blowfish, RC2, IDEA, XOR, MD2, MD4, JWT, and sha224 are skipped;
- positives must fire the corresponding rule scope, and negatives must clearly use a safe alternative such as SHA-256, AES-GCM, `os.urandom`, or `create_default_context`.

These 208 rule fixtures are all `verified=false`. They are tool-regression and finding-coverage data, not real vulnerability positives; redistribution must respect the licence of the Semgrep source lines.

5.3 Secure-library hedging and the reference layer

A hard problem in R4/R5 was that secure library code was heavily over-reported. R6 adds negatives from many secure libraries, including cryptography, PyNaCl, itsdangerous, PyJWT, PyCryptodome, passlib, argon2-cffi, ecdsa, google-auth, and oscrypto. The goal is not "the project is secure, so the function is safe" but a per-function check of parameters, purpose, key origin, and context.

Of the 15 fixed-commit audit candidates in `result.txt`, 7 were marked `secure_reference` and written to a separate reference layer; they keep `verified=false` and `training_eligible=false` and are used only for fix suggestions, safe-code comparison, and later manual promotion, not for training. The other 8 were recommended for exclusion and were not merged.

5.4 The real candidate pool: from advisory to function-level code pair

R6's data engineering is not "download a dataset" but a traceable pipeline of candidate building, screening, code completion, review, and merging:

1. collect candidates from GHSA, OSV/PyPI, PyPA, PyCode-Vul, and local JSON/JSONL;
2. screen by CWE, rule, package name, commit, repo, and keyword; de-duplicate into a candidate pool;
3. fetch vulnerable/fixed code pairs via the fix commit; the 366 that pass `ast.parse` are better suited to function-level training;
4. rank by evidence strength into P1/P2/P3; P1/P2 enter the priority review queue;
5. run strict binary pre-review with a strong model, or send to browser-based human review;
6. only Confirm/Reject conclusions enter `real_verified_detect.jsonl` and `real_verified_triage.jsonl`;
7. merge the R5 base with the builder, producing train/val/test and the SHA-256 manifest.

The key offline snapshot:

This snapshot is itself an important fact: R6's bottleneck is not "no advisories" but the gap between an advisory and a trainable function pair, in code, commit, repository, and human evidence.

5.5 Strong-model pre-review: budget, model choice, and strict binary output

Brute-force full review is not realistic, so the project estimates cost before calling. The actual K3 usage for the Class A function-candidate batch was:

The estimation stage used a conservative assumption of about CNY 33.59 for high-reasoning output; the actual output was far below the assumed ceiling. This gap shows why "estimate first, call second, report actual usage third" is a necessary flow; cost must not be guessed from candidate count.

The review prompt uses a strict binary format:

Table 13: R6 candidate-pool snapshot

Metric	Value
On-disk external advisory sources	GHSA approx. 380k JSON, OSV-PyPI approx. 26k JSON, PyPA 7,721 YAML, Bandit, Semgrep, Trail of Bits rules
Per-source harvester output	GHSA 534, OSV-PyPI 843, PyPA 706, PyCode-Vul 389
Merge and de-duplicate	2,718 merged, 1,861 selected after de-duplication
P1 / P2 / P3	5 / 2 / 1,854
With code pair	400 records, of which 366 parse at both ends with `ast.parse`
Missing code only	549 records have a fix commit
Missing commit and repo	703 records need a package-to-repo mapping first
Class B training volume	331 parseable real production code pairs, no advisory
Full P3 strong-model estimate	K3 high approx. CNY 975.18, K2.7-code high approx. CNY 267.01; not affordable to run blind on everything

Table 14: K3 real-candidate review cost

Item	Value
Candidates	60
prompt tokens	240,546
completion tokens	26,995
Actual input cost	CNY 4.81092
Actual output cost	CNY 2.69950
Actual total cost	**CNY 7.51042**
Model / reasoning effort	`kimi-k3` / `high`
Price basis	input CNY 20 per million tokens, output CNY 100 per million tokens

- only `confirmed` or `rejected` is allowed;
- insufficient evidence is forced to `rejected`; no uncertain option is introduced;
- API, parsing, or schema errors are recorded as error and not imported;
- imported records keep `review_mode=model`, `model=kimi-k3`, `model_confidence`, and `source_audit`;
- the 60 K3 results must not be described as human ground truth.

The import report shows: 60 review lines; on incremental import 48 were already reviewed and 1 was newly imported; 8 lacked vulnerable code and 3 had a vulnerable snippet that is not valid Python; all were skipped and never entered the verified export. The final K3 contribution is 23 detect + 37 triage.

5.6 The browser-based human review platform

To reduce manual effort, only the uncertain and sampled parts go to browser review. The final UI is deliberately simplified to two buttons:

- **Confirm**: exports one detect positive and one triage Confirm;
- **Reject**: exports one triage Reject;
- uncertain, scope-out, and license fields are removed;

- Confirm requires a target rule, target CWE, an explanation, and two Python snippets that pass `ast.parse`;
- Reject requires a vulnerable snippet and an explanation;
- the reviewer may fix the snippet in the browser, but cannot enter samples unsupported by the source.

Each verdict is the source of truth, stored at:

`data/round6/review/web/<reviewer>/labels/<verdict>/<candidate_id>.json`

The application rebuilds `real_verified_detect.jsonl` and `real_verified_triage.jsonl` from these files. The final 19 human-reviewed records are 7 detect + 12 triage; one candidate was excluded because its code exceeded the allowed length. Human labels and model labels share the same export schema but carry different provenance fields.

5.7 Pre-training freeze and reproducibility constraints

Before training, R6 generates `r6_manifest.json`, freezing the source files and the SHA-256 of train/val/test. The key artifacts are:

Table 15: R6 uploaded-artifact hashes

Artifact	Rows	SHA-256
train	3115	`139966e5217d4b2976db7a13f86d70ecb8aad09adc1198092f782e028041bd4d`
val	347	`6b6797b5fb1da66c797f2efe28dc7559d124b2cd4adb5f75496a40f07d397f4b`
test	660	`1549c1d7d5c8484aafe677ddf83ccfbdf6912e6052d1e910d49f45ed66c7d64c`

After freezing, train/val/test must not be modified. Evaluation uses `validation_frozen/`, which is invisible during training.

6 R6 Training and Reproducibility Information

6.1 Model and platform

Table 16: R6 model and workspace

Item	Value
Deployed model ID	`qwen3-4b-instruct-2507-5f8261ad123d`
Fine-tune job	`ft-202610051556-cc87`
Fine-tune output model	`qwen3-4b-instruct-2507-ft-202610051556-cc87`
Base	`qwen3-4b-instruct-2507`, fresh base
Bailian workspace	`ws-avxkjb2tq5lq1gwm`
Training platform	Alibaba Cloud Bailian cn-beijing
Inference path	detect direct; triage appends `TRIAGE_PURPOSE_GUIDE`

6.2 Hyperparameters and configuration differences

The guide written before R6 recorded planned parameters, while the platform finally reported actual runtime parameters. The two agree on LoRA structure but differ on batch and max length:

Table 17: R6 planned hyperparameters vs platform report

Parameter	R6 guide plan	Platform / eval-report value
LoRA rank	16	16
LoRA alpha	32	32
LoRA dropout	0.005	0.005
learning rate	1e-4	1e-4
epochs	3	3
batch size	16	128
max length	2048	4096
scheduler	cosine	cosine

This report uses the platform-reported parameters in the final model description while keeping the planned values as a reproducibility difference. The difference also reminds us that an experiment report must record both "planned configuration" and "actual runtime configuration", not only one of them.

7 R6 Evaluation Results

7.1 Evaluation suites

R6 uses four evaluation suites:

1. frozen holdout: de-leaked 19 detect + 36 triage;
2. 18 real triage probes: 12 Confirm + 6 Reject;
3. R6 design test split: 602 detect + 58 triage, same source as training, valid only as a relative reference;
4. capability matrix and secure-library anti-false-positive: 13 rules with one positive and one negative each, plus 10 secure patterns.

7.2 Frozen holdout

Table 18: R6 frozen holdout results

Slice	n	recall	precision	F1	CWE acc	Note
detect	19	0.714	0.500	0.588	0.400	tp/fp/fn/tn = 5/5/2/7
triage	36	1.000*	–	–	–	FPR 0.125, accuracy 0.889, only 4 Confirm

* Confirm recall. With only 4 positives this is a count, not a statistical estimate.

Of the 12 safe detect samples, 5 were over-reported; of the 7 vulnerability positives, 2 were missed. The 4 triage false positives were `Crypto_IO__PBES__PBES2_encrypt`, `passlib_cisco_pix`, `passlib_sun_md5_crypt`, and `r5v-011-tneg`. All 4 true Confirm cases were hit, but because there are only 4 positives, Confirm recall 1.000 is only a count and cannot serve as a statistical acceptance test.

7.3 The 18 real triage probes

Table 19: R6 18-probe results

Metric	Value
n	18 (12 Confirm / 6 Reject)
Confirm recall	0.500
FPR	0.000
accuracy	0.667

All six errors are Confirm → Reject; no Reject was over-reported as Confirm. The errors fall into two classes:

1. **explanation-verdict contradiction (3 cases)**: the explanation concludes `-> Confirm` but the JSON `verdict` outputs Reject, for example MD5 password comparison, a weak-random token, and an MD5 signature;
2. **hallucinated absent security context (3 cases)**: the model explains a pure cryptographic function as a "cache key", "test fixture generating an ETag", or "cache consistency", while the input code has none of that context.

This shows that R6's most serious regression is not a plain recall miss but an unstable triage decision chain: explanation ability is still present, but the output field and the security-purpose reasoning are not fully aligned.

7.4 R6 design test split

Table 20: R6 test-split results

Slice	n	recall	precision	F1 / FPR
detect	602 (325 pos / 277 neg)	0.859	0.550	F1 0.671
triage	58 (28 Confirm / 30 Reject)	0.571*	–	FPR 0.200, accuracy 0.690

* Confirm recall.

detect is `tp/fp/fn/tn = 279/228/46/49`, with CWE accuracy 0.724. The test split shares a source with training, and the detect ground truth contains PyCode-Vul model-prediction noise, so this suite can only serve as a relative reference.

7.5 13-rule capability matrix

Vulnerability detection is 11/13, CWE correctness is 6/13, and safe-not-flagged is 10/13. The model catches obvious patterns but collapses several CWEs into CWE-327, CWE-259, or CWE-798, and still over-reports safe patterns.

7.6 Secure-library anti-false-positive

Of 10 secure patterns, 8 were not over-reported, FPR 0.20. The false positives are:

- `cryptography.hazmat...AESGCM(key).encrypt(os.urandom(12), ...)` judged vulnerable CWE-327;
- `nacl.secret.SecretBox(key).encrypt(msg)` judged vulnerable CWE-327.

7.7 Acceptance gates

8 R6 Versus R5: What Recovered, What Did Not

8.1 Recovery

This comparison is directional, not strictly matched: the R5 and R6 evaluation slices are not identical, and R5 itself had run-to-run variance. R6 demonstrates that a full fresh-base retrain can recover from failure and learn a weak signal for obvious cryptographic patterns, but it does not demonstrate production-grade precision or real-world Confirm recall.

8.2 Remaining problems

1. **CWE routing is a clear weakness**: frozen CWE accuracy 0.400, matrix 6/13, with a tendency to collapse into CWE-327, CWE-259, and CWE-798;
2. **triage Confirm recall regressed**: 0.500 on the 18 probes, 0.571 on the test split, with explanation-verdict contradictions;
3. **secure libraries are still over-reported**: AESGCM and `nacl.SecretBox` were wrongly flagged;
4. **too few real positives**: 30 is below the 50–100 gate and cannot support a real-world crypto generalisation claim;
5. **insufficient frozen-set statistical power**: detect 19 and triage 36 with only 4 Confirm;
6. **prompt-alignment problem**: training data strips `TRIAGE_PURPOSE_GUIDE` while inference appends it; the guide ends with "non-security purpose → Reject", which may pull the verdict toward Reject;
7. **training-distribution generalisation problem**: too many added Reject, bneg, legacy, test fixture, and cache samples; the model over-generalises these contexts onto real vulnerabilities;
8. **residual leakage**: the frozen set is clean relative to the optimiser-visible set, but the merged train/val/test still contains inherited duplicates.

9 Conclusion, Limitations, and the R7 Roadmap

9.1 Conclusion

R6 is an honest, reproducible research baseline. It shows that:

- static rules + a small domain model + structured output can run end to end;
- a fresh-base full merge retrain can recover the R5 collapse;
- the model can learn explicit patterns such as MD5, DES, RC4, ECB, and weak TLS;
- data provenance, review mode, SHA-256 freezing, and de-leaked evaluation can be engineered;
- real-data bottleneck, sample volume, and contextual labels are constraints more critical than "training longer".

It does not show that:

- production-grade precision, Confirm recall, or CWE routing is achieved;
- unknown cryptographic vulnerabilities can be discovered by generalisation;
- false positives on secure library code are robustly controlled;
- model review equals human ground truth;
- the merged training data is leak-free.

9.2 Main limitations

First, there are only 30 real crypto positives, concentrated in CWE-295 and CWE-208. Per-class results on rare CWEs are unstable and cannot be hidden behind aggregate scores.

Second, the frozen set is very small. triage has only 4 Confirm cases, so a single prediction flip changes recall noticeably; absolute counts and confidence intervals should be reported, and 1.000 must not be treated as proof of capability.

Third, general PyCode-Vul labels are mixed with crypto-specific truth. Of 2,037 detect positives only 434 belong to target crypto CWEs, so the aggregate detect metric cannot measure crypto generalisation.

Fourth, strong-model review is model evidence, not human evidence. The 60 K3 records and 19 human records must be counted separately; K3 can reduce workload but cannot replace independent ground truth.

Fifth, train/inference prompt inconsistency directly affects the verdict. R6's three "correct explanation but opposite verdict" cases are likely related to this, and later work must either align the prompt or add a structured field-consistency check.

Sixth, residual AST duplicates show that data inheritance still carries risk. Future work must split stratified by repo/commit and re-run the exact/AST audit after every merge.

9.3 R7 roadmap

1. **Scale real positives to 50–100**: continue GHSA/OSV/PyPA → fix commit → parent source → function rebuild → strong-model/human review; report coverage by advisory, repo, CWE, and rule, not just total count.
2. **Build a true external test set**: use time splits, repository splits, and fixed commits not involved in rule design; report bootstrap confidence intervals rather than a single point score.
3. **Add a train-only crypto-focus sampler**: set minimum sampling quotas by target CWE and rule family, protect rare classes such as CWE-321/329/916/256/208, and never touch val/test.
4. **Align train and inference prompts**: either add the purpose guide during training or remove it at inference; also add JSON-schema validation and an "explanation consistent with verdict" check.
5. **Improve secure negative quality**: keep extracting from fixed-commit secure libraries, but check the call context first; prioritise AESGCM, SecretBox, legacy-format, and test/cache/ETag contexts so that weak-primitive usage is not over-generalised.

6. **Improve CWE targets**: build per-CWE precision/recall for each target CWE and add CWE hierarchy or rule-mapping assistance so the model does not freely collapse to CWE-327.
7. **Separate synthetic, rule-fixture, and real-record evaluation layers**: rule fixtures for tool regression, real records for quality evaluation, synthetic samples for coverage; the three must not share one headline.
8. **Upgrade the base model or continue pre-training**: only after data quality passes the gate should a stronger base, a longer context, or domain continued pre-training be considered; a base-model upgrade cannot compensate for wrong labels.
9. **Build an annotation quality mechanism**: double-review a small sample and measure agreement; perform error analysis on K3-versus-human disagreements; gradually convert high-confidence model labels into sampled labels.
10. **Continue leakage and licence governance**: Semgrep rule fixtures follow the Semgrep Rules License v1.0 and Bandit follows Apache-2.0; retain provenance and licence metadata on publication.

10 Review Q&A and Interview Points

10.1 Frequently asked questions

Why cryptographic code security?

Cryptographic code has clear APIs and clear CWEs, and it is strongly context-dependent. It tests both static rules and the LLM's weaknesses in security semantics, hallucination, and data quality.

Why combine Semgrep and an LLM?

Semgrep provides high-recall, locatable, interpretable candidates; the LLM provides purpose judgement and false-positive filtering. Semgrep alone produces valid false positives from legacy compatibility, caches, and test fixtures; an LLM alone lacks stable location and reproducible rules.

Why fine-tune a small model instead of using a large-model API directly?

A small model is cheaper, deployable, and researchable for domain adaptation, and it is easy to combine with static rules. The project puts the large model in the data-review stage, not in the production inference path.

What are the most important technical findings?

First, fresh-base full retraining is more stable than incremental continued training; second, the data-quality gate matters more than sample count; third, real crypto positives and human/model provenance tagging are the core bottleneck; fourth, prompt alignment directly affects the triage verdict; fifth, small positive counts make recall estimates very unstable.

Was R6 a failure or a success?

R6 successfully recovered the R5 detect collapse and formed a reproducible data and evaluation pipeline, but it did not pass the precision, FPR, and probe-accuracy gates, so it can only be defined as a temporary research baseline, not a successfully shipped production model.

10.2 Things to watch during the experiments

- never mix metrics across rounds; every number must state its round, slice, and sample size;
- always report the positive count alongside Confirm recall;

- never write `verified=false` rule fixtures as real vulnerabilities;
- never write K3 labels as human labels;
- never claim zero leakage;
- never treat the R6 test-split borderline precision as evidence of production readiness;
- verify the manifest hashes before training and do not modify train/val/test afterwards;
- store both planned and actual hyperparameter values;
- check regulation, licence, and provenance for every real-data addition.

A Reproduction Commands and Key Files

A.1 Build and freeze

```
python3 scripts/build_round5_final_dataset.py --emit
python3 scripts/harvest_real_crypto_candidates.py \
  --advisories data/external/advisory-database/advisories
python3 scripts/screen_real_crypto_candidates.py --emit
python3 scripts/rebuild_candidate_functions.py --dry-run
GITHUB_TOKEN=ghp_xxx python3 scripts/rebuild_candidate_functions.py
python3 scripts/audit_real_crypto_with_llm.py --estimate-only
python3 scripts/annotate_real_crypto.py \
  --candidates data/round6/candidates/real_crypto_candidates_uncertain.jsonl \
  --reviewer reviewer-kimi --open
python3 scripts/build_round6_final_dataset.py --emit
python3 scripts/audit_dataset_leakage.py --write-frozen
python3 scripts/build_dataset_metadata.py
```

A.2 Evaluation

```
python3 scripts/run_round6_eval.py
python3 scripts/run_round6_test_split.py
python3 scripts/run_round6_capability.py
```

A.3 Key artifacts

- `data/round6/eval/RESULTS.md`: full R6 evaluation conclusion;
- `data/round6/eval/benchmark_r6.json`: frozen detect + triage per-record predictions;
- `data/round6/eval/triage_probe_results_r6.json`: the 18 probes and explanations;
- `data/round6/eval/test_split_r6.json`: per-record predictions over the 660-record test split;
- `data/round6/eval/capability_r6.json`: the 13-rule matrix and secure-library probe;
- `reports/data_quality/model_audit_k3_cost.json`: real K3 cost;
- `reports/data_quality/model_audit_k3_import.json`: K3 import and skip reasons;
- `data/round6/final/r6_manifest.json`: source files and artifact hashes;
- `data/round6/validation_frozen/manifest.json`: frozen evaluation set and de-leak notes;
- `reviewer/README.md`: browser binary-review instructions.

B Glossary

Table 21: R6 13-rule capability matrix

Rule	Expected CWE	Vuln. detected	Predicted CWE	Safe not flagged
CRYPTO-001 MD5	CWE-327	✓	CWE-327	✓
CRYPTO-002 SHA-1	CWE-327	×	–	✓
CRYPTO-003 DES	CWE-327	✓	CWE-327	✓
CRYPTO-004 RC4	CWE-327	✓	CWE-327	✓
CRYPTO-005 AES-ECB	CWE-327	✓	CWE-327	✓
CRYPTO-006 predictable IV	CWE-329	✓	CWE-327	✓
CRYPTO-007 hard-coded key	CWE-321	✓	CWE-327	×
CRYPTO-008 weak randomness	CWE-338	×	–	✓
CRYPTO-009 weak key length	CWE-326	✓	CWE-326	✓
CRYPTO-010 insecure TLS	CWE-326	✓	CWE-326	✓
CRYPTO-011 plaintext password	CWE-256	✓	CWE-259	×
CRYPTO-012 weak KDF parameters	CWE-916	✓	CWE-798	×
CRYPTO-013 timing comparison	CWE-208	✓	CWE-259	✓

The capability fixture labels CRYPTO-010 with expected CWE-326, although the rule table maps it to CWE-295; this fixture/spec mismatch is itself a data-consistency item to fix in R7.

Table 22: R6 acceptance gates

Gate	Target	R6 status
detect precision	> 0.54	× frozen 0.500; test split 0.550, borderline
domain FPR	≤ 0.06	× frozen 0.125, test split 0.200, secure libraries 0.20
triage Confirm recall	report only	frozen has only 4 Confirm, no statistical meaning
18-probe accuracy	≥ 0.95	× 0.667
real-repository FPR	no worse than R3/R4	× unresolved

Table 23: Directional R5 vs R6 comparison

Metric	R5	R6	Interpretation
detect F1	0.000	0.588 frozen / 0.671 test	recovered from the detect collapse
detect recall	0.000	0.714 frozen / 0.859 test	basic detection ability is back
triage FPR	0.368	0.125 frozen	frozen-set false positives fell but remain above target
triage accuracy	0.658	0.889 frozen	improved on the frozen set, but the composition differs
Confirm recall	1.000 on 3 positives	0.500 on 18 probes	shows small-positive recall cannot be extrapolated

Term	Meaning
SAST	Static application security testing; analyse source without running the program
detect	Input code only; judge vulnerability and CWE
triage	Input code and a finding; decide Confirm/Reject
CWE	Common Weakness Enumeration
CSPRNG	Cryptographically secure pseudo-random number generator
AEAD	Authenticated encryption providing both confidentiality and integrity
KDF	Key-derivation function from a password or master key
LoRA / QLoRA	Low-rank adaptation / quantised low-rank adaptation
fresh-base retrain	Retrain from the original base on the full merged dataset
catastrophic forgetting	New training degrades old abilities
FPR	Fraction of true negatives wrongly judged positive
verified=false	Not yet independently verified for authenticity
`review_mode`	Review provenance: `manual` or `model`